

Cloud Security Study Plan — Phase 5 Progress Log

Docker Study Host on AWS EC2 (ap-southeast-5, Terraform-managed)

Part 1 — Terraform (Infrastructure as Code)

1.1 Setup Context

Initial manual EC2 setup was done in us-east-1, then fully migrated to Terraform-managed infrastructure in ap-southeast-5 (AWS Malaysia region) for lower latency and IaC practice ahead of schedule (originally planned for Weeks 21-22).

1.2 Install Terraform (Windows)

```
winget install Hashicorp.Terraform
terraform -version
```

Result: Terraform v1.15.8 installed successfully.

1.3 Project Structure

```
terraform-docker-host\
  provider.tf
  security_group.tf
  ec2.tf
  lambda.tf
  lambda\
    lambda_function.py
```

1.4 Resources Defined

- provider.tf — AWS provider, region ap-southeast-5, profile "kelvin"
- security_group.tf — Security group allowing SSH (port 22) and later Flask app (port 5000) from caller's current public IP only, auto-detected via the http data source (checkip.amazonaws.com)
- ec2.tf — t3.micro instance, Amazon Linux 2023 AMI (ECS-optimized variant), 30GB gp3 root volume, auto-generated TLS key pair saved locally as a .pem file
- lambda.tf — Lambda function (Python 3.12) that auto-stops the EC2 instance after 4 hours of runtime, IAM execution role + inline policy, EventBridge Scheduler rule running every 15 minutes to check runtime
- lambda/lambda_function.py — Runtime-check logic; reads INSTANCE_ID and MAX_RUNTIME_HOURS from environment variables (injected by Terraform, no hardcoding)

1.5 Commands Used

Initialize the working directory

```
cd "C:\Users\Kelvin\Desktop\Cloud Sec\terraform-docker-host"
terraform init
```

Preview planned changes

```
terraform plan
```

Apply / create infrastructure

```
terraform apply
```

First apply failed with: "InvalidBlockDeviceMapping: Volume of size 8GB is smaller than snapshot, expect size >= 30GB". Fixed by changing root_block_device volume_size from 8 to 30 in ec2.tf, then re-applied successfully (4 resources added: instance + dependents).

Re-apply after home IP changes

Because the security group locks SSH/port 5000 to the IP captured at apply time, terraform apply must be re-run whenever the local public IP changes (e.g., different network) — Terraform detects the new IP automatically via the http data source and updates the security group rule.

1.6 Key Outputs

- instance_id — EC2 instance identifier (e.g., i-0a4f8dbebb5cea5f2)
- instance_public_ip — changes on every stop/start since no Elastic IP is attached
- ssh_command — ready-to-use SSH command with the correct key filename

1.7 Instance Lifecycle Commands (AWS CLI, run locally)

Stop the instance (preserves disk, Docker, images, containers)

```
aws ec2 stop-instances --instance-ids i-0a4f8dbebb5cea5f2 --region ap-southeast-5
```

Start the instance

```
aws ec2 start-instances --instance-ids i-0a4f8dbebb5cea5f2 --region ap-southeast-5
```

Get the current public IP after starting

```
aws ec2 describe-instances --instance-ids i-0a4f8dbebb5cea5f2 \
  --region ap-southeast-5 \
  --query "Reservations[0].Instances[0].PublicIpAddress" --output text
```

1.8 us-east-1 Manual Resource Cleanup (pre-Terraform resources)

Before rebuilding in ap-southeast-5, the original manually created us-east-1 resources were torn down via AWS CLI:

```
aws ec2 terminate-instances --instance-ids <id> --region us-east-1
aws lambda delete-function --function-name ec2-auto-stop-docker-host --region us-east-1
aws scheduler delete-schedule --name ec2-runtime-check --region us-east-1
aws iam detach-role-policy --role-name <role> --policy-arn <arn>
aws iam delete-role-policy --role-name <role> --policy-name ec2-stop-permission
aws iam delete-role --role-name <role>
aws iam delete-policy --policy-arn <arn>
aws ec2 delete-security-group --group-id <sg-id> --region us-east-1
aws ec2 delete-key-pair --key-name docker-study-key --region us-east-1
```

1.9 Windows Key File Permissions (required before SSH)

PowerShell (not CMD) syntax — needed because %username% does not expand in PowerShell:

```
icacls docker-study-key-my5.pem /reset
icacls docker-study-key-my5.pem /inheritance:r
```

```
icacls docker-study-key-my5.pem /grant:r "${env:username}:R"  
icacls docker-study-key-my5.pem # verify – should show only your user with (R)
```

1.10 Adding a New Ingress Rule (example: exposing Flask on port 5000)

Added a second ingress block to `security_group.tf`, below the existing SSH block and above egress, then re-ran terraform apply — no need to stop the instance or interrupt SSH sessions, since security group changes apply immediately and independently of the instance itself.

```
ingress {  
  description = "Flask app from my IP"  
  from_port   = 5000  
  to_port     = 5000  
  protocol    = "tcp"  
  cidr_blocks = [local.my_ip_cidr]  
}
```

Part 2 — Docker (Week 17–18)

2.1 Connect to the Host

```
ssh -i docker-study-key-my5.pem ec2-user@<PUBLIC_IP>
```

2.2 Install Docker (Amazon Linux 2023)

```
sudo dnf update -y  
sudo dnf install -y docker  
sudo systemctl enable --now docker  
sudo usermod -aG docker ec2-user  
# then exit and reconnect for group change to take effect
```

Note: the AMI used was the ECS-optimized variant of Amazon Linux 2023, which auto-launches an `amazon-ecs-agent` container on boot (visible in `docker ps -a`). It exits with an error since no ECS cluster is configured — harmless, and does not affect Docker usage.

2.3 Verify Installation

```
docker --version  
docker run hello-world
```

2.4 Run an Interactive Container

```
docker run -it ubuntu bash  
cat /etc/os-release  
exit
```

Demonstrated container isolation — the Ubuntu container has a completely separate filesystem and OS identity from the Amazon Linux host, despite running on the same physical instance.

2.5 Inspect Containers and Images

```
docker ps -a  
docker images
```

2.6 Build a Custom Image — "cowsay" example

```
mkdir ~/first-image && cd ~/first-image
cat > Dockerfile << 'EOF'
FROM ubuntu:latest
RUN apt-get update && apt-get install -y cowsay
ENV PATH="/usr/games:${PATH}"
CMD ["cowsay", "Hello from my first Docker image!"]
EOF
docker build -t my-first-image .
docker run my-first-image
```

Encountered and resolved a real packaging issue: cowsay installs to /usr/games, which is not on the default non-interactive PATH. Fixed with an ENV instruction in the Dockerfile.

2.7 Build and Run a Flask Web App

```
mkdir ~/flask-app && cd ~/flask-app

# app.py
from flask import Flask
app = Flask(__name__)

@app.route("/")
def hello():
    return "Hello from my Dockerized Flask app, Kelvin!"

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)

# Dockerfile
FROM python:3.12-slim
WORKDIR /app
COPY app.py .
RUN pip install flask
EXPOSE 5000
CMD ["python", "app.py"]

docker build -t flask-app .
docker run -d -p 5000:5000 --name flask-app-running flask-app
curl http://localhost:5000 # internal test
```

Note: host="0.0.0.0" in app.py is required — the Flask default of 127.0.0.1 only listens inside the container and would not be reachable via port mapping.

After opening port 5000 in the security group (see section 1.10) and re-running terraform apply, the app was successfully accessed from a browser at http://<PUBLIC_IP>:5000 — confirmed working externally.

2.8 Container Lifecycle Management

```
docker stop flask-app-running # stops the container, image and data remain
docker start flask-app-running # restarts without rebuilding
```

2.9 Status

Week 17-18 (Docker) is complete: image pulling, container isolation, custom image builds, Dockerfile authoring, port mapping, and external browser access all demonstrated end-to-end on the Terraform-provisioned EC2 host.